

COMPTE RENDU

Cryptographie appliquée & Blockchain - TP2 - Chiffrement à Clé Secrète
4e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants : HATHOUTI Mohammed Taha

JIDAL Ilyas

KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : Mme. FADI

Date : 3 octobre 2026

Table des matières

Objectifs	2
1 Task 1 : Analyse fréquentielle	3
1.1 Étape 1 — Statistiques avec <code>freq.py</code>	3
1.2 Étape 2 — Premières lettres : trigrammes et bigrammes	5
1.3 Étape 3 — Lettres restantes : complétion de mots	9
1.4 Clé trouvée et vérification	12
2 Task 2 : Chiffrement avec différents algorithmes et modes	13
2.1 Étape 1 — Fichier clair	13
2.2 Étape 2 — Paramètres utilisés	13
2.3 Étape 3 — Chiffrement avec quatre algorithmes / modes	13
2.4 Étape 4 — Taille des fichiers	14
2.5 Étape 5 — Contenu des fichiers chiffrés	14
2.6 Étape 6 — Déchiffrement et vérification	15
2.7 Observations	15
3 Task 3 : Modes de chiffrement ECB et CBC sur une image	16
3.1 Étape 1 — Image de départ	16
3.2 Étape 2 — Chiffrement en ECB et en CBC	16
3.3 Étape 3 — Tentative d’ouverture directe	17
3.4 Étape 4 — Reconstruction des images	17
3.5 Étape 5 — Affichage et conversion en PNG	17
3.6 Observations	18
4 Task 4 : Padding	19
4.1 Étape 1 — Création de trois fichiers de 5, 10 et 16 octets	19
4.2 Étape 2 — Chiffrement avec les modes ECB, CBC, CFB et OFB	19
4.3 Étape 3 — Comparaison des tailles	20
4.4 Étape 4 — Observation du padding avec <code>-nopad</code>	21
4.5 Observations	22
5 Task 5 : Propagation d’erreur dans un chiffré corrompu	22
5.1 Étape 1 — Fichier clair d’au moins 1000 octets	22
5.2 Étape 2 — Chiffrement avec les modes ECB, CBC, CFB et OFB	22
5.3 Étape 3 — Corruption d’un bit du 55 ^e octet	23
5.4 Étape 4 — Déchiffrement des fichiers corrompus	23
5.5 Étape 5 — Nombre d’octets qui diffèrent du clair d’origine	24
5.6 Étape 6 — Aperçu du clair obtenu (ECB et OFB)	24
5.7 Observations	25
6 Difficultés rencontrées	26
7 Conclusion	26

Objectifs

Ce TP (*Secret-Key Encryption Lab*, SEED Labs) a pour but de se familiariser avec le chiffrement à clé secrète et avec quelques attaques courantes contre celui-ci : chiffrement par substitution et analyse fréquentielle, modes de chiffrement (ECB, CBC, CFB, OFB), padding, vecteur d'initialisation (IV) et programmation avec la bibliothèque crypto d'OpenSSL. Il montre aussi que de nombreuses vulnérabilités viennent non pas des algorithmes eux-mêmes, mais de la façon dont les développeurs les utilisent.

Chiffrement : $C = E_k(P)$ Déchiffrement : $P = D_k(C)$ avec $D_k(E_k(x)) = x$

Pour un chiffrement par substitution monoalphabétique, la clé k est une permutation π des 26 lettres de l'alphabet : chaque lettre claire p est toujours remplacée par la même lettre chiffrée $\pi(p)$, et le déchiffrement utilise la permutation inverse π^{-1} .

1 Task 1 : Analyse fréquentielle

Retrouver, à partir du fichier *ciphertext.txt* (fourni dans *Labsetup.zip*), la clé de substitution et le texte original, un article en anglais chiffré par substitution monoalphabétique.

1.1 Étape 1 — Statistiques avec freq.py

Dans Labsetup/Files, on lance `python3 freq.py` : le programme affiche les 1-grammes, 2-grammes et 3-grammes les plus fréquents du texte chiffré.

```
1 hathouti@Taha - inspiron - 16: ~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ python3 freq.py  
2 -----  
3 1-gram (top 20):  
4 n: 488  
5 y: 373  
6 v: 348  
7 x: 291  
8 u: 280  
9 q: 276  
10 m: 264  
11 h: 235  
12 t: 183  
13 i: 166  
14 p: 156  
15 a: 116  
16 c: 104  
17 z: 95  
18 l: 90  
19 g: 83  
20 b: 83  
21 r: 82  
22 e: 76  
23 d: 59  
24 -----  
25 2-gram (top 20):  
26 yt: 115  
27 tn: 89  
28 mu: 74  
29 nh: 58  
30 vh: 57  
31 hn: 57  
32 vu: 56  
33 nq: 53  
34 xu: 52  
35 up: 46  
36 xh: 45  
37 yn: 44  
38 np: 44  
39 vy: 44  
40 nu: 42
```

```

41 qy: 39
42 vq: 33
43 vi: 32
44 gn: 32
45 av: 31
46 -----
47 3-gram (top 20):
48 ytn: 78
49 vup: 30
50 mur: 20
51 ynh: 18
52 xzy: 16
53 mxu: 14
54 gnq: 14
55 ytv: 13
56 nqy: 13
57 vii: 13
58 bxh: 13
59 lvq: 12
60 nuy: 12
61 vyn: 12
62 uvy: 11
63 lmu: 11
64 nvh: 11
65 cmu: 11
66 tmq: 10
67 vhp: 10

```

1-gramme	Occ.	2-gramme	Occ.	3-gramme	Occ.
n	488	yt	115	ytn	78
y	373	tn	89	vup	30
v	348	mu	74	mur	20
x	291	nh	58	ynh	18
u	280	vh	57	xzy	16
q	276	hn	57	mxu	14
m	264	vu	56	gnq	14
h	235	nq	53	ytv	13
t	183	xu	52	nqy	13
i	166	up	46	vii	13

TABLE 1 – Les dix premiers n-grammes du texte chiffré (Étape 1)

Sur les 3931 lettres du texte, les plus fréquentes sont comparées à la distribution de

l'anglais :

Lettre chiffrée	Occurrences	Fréq. (%)	Clair supposé	Fréq. anglais (%)
n	488	12,4	E	12,7
y	373	9,5	T	9,1
v	348	8,9	A	8,2
x	291	7,4	O	7,5
u	280	7,1	N	6,7
q	276	7,0	S	6,3
m	264	6,7	I	7,0
h	235	6,0	R	6,0
t	183	4,7	H	6,1
i	166	4,2	L	4,0

TABLE 2 – Fréquences des lettres du texte chiffré et lettres claires supposées

1.2 Étape 2 — Premières lettres : trigrammes et bigrammes

Les lettres de fréquences voisines (O, N, S, I, R, H) ne se départagent pas par leur seule fréquence : on s'appuie sur les trigrammes et bigrammes.

Chiffré	Clair	Indice
n, y, t	E, T, H	ytn (78 occurrences, trigramme le plus fréquent) → THE
v, u, p	A, N, D	vup (30) → AND
m, r	I, G	mur (20) → ING ; mu (74) → IN
h	R	ynh (18) → TER ; nh et hn → ER et RE
x, z	O, U	xzy (16) → OUT ; mxu (14) → ION
q	S	nq (53) et qy (39) → ES et ST
i	L	vii (13) → ALL

TABLE 3 – Treize lettres déduites des n-grammes

On applique ces 13 correspondances avec **tr**, en écrivant le clair en majuscules pour le distinguer des lettres encore chiffrées :

```
1 hathouti@Taha - inspiron - 16: ~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ tr 'nytvupmrhxxqi' 'ETHANDIGROUSL' < ciphertext.
  txt > out.txt
```

2 hathouti@Taha -inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
 Labsetup/Files\$ cat out.txt
 3 THE OSaARS TURN ON SUNDAd lHIaH SEEcS AgOUT RIGHT AbTER THIS LONG
 STRANGE
 4 ALARDS TRIE THE gAGGER bEELS LISe A NONAGENARIAN TOO
 5
 6 THE ALARDS RAaE lAS gOOsENDED gd THE DEcISE Ob HARfEd lEINSTEIN AT
 ITS OUTSET
 7 AND THE AeeARENT IceLOSION Ob HIS bILc aOceAnd AT THE END AND IT
 lAS SHAeED gd
 8 THE EcERGENaE Ob cETOO TICES Ue gLAasGOlN eOLITiaS ARcaANDd
 AaTifISc AND
 9 A NATIONAL aONfERSATION AS gRIEb AND cAD AS A bEfer DREAc AgOUT
 lHETHER THERE
 10 OUGHT TO gE A eRESIDENT lINbREd THE SEASON DIDNT oUST SEEc EkTRA
 LONG IT lAS
 11 EkTRA LONG gEaAUSE THE OSaARS lERE cOfED TO THE bIRST lEEsEND IN
 cARaH TO
 12 AfOID aONbLIaTING lITH THE aLOSING aEREcONd Ob THE lINTER OlDceIaS
 THANsS
 13 edEONGaHANG
 14
 15 ONE gIG jUESTION SURROUNDING THIS dEARS AaADEcd ALARDS IS HOl OR Ib
 THE
 16 aEREcONd lILL ADDRESS cETOO ESeEaIALld AbTER THE GOLDEN GLOgES
 lHIaH gEaAcE
 17 A oUGILANT aOcINGOUT eARTd bOR TICES Ue THE cOfEcENT SeEARHEADED gd
 18 eOlERbUL HOLLdLOOD lOcEN lHO HELEED RAISE cILLIONS Ob DOLLARS TO
 bIGHT SEKUAL
 19 HARASScENT AROUND THE aOUNTRd
 20
 21 SIGNALING THEIR SUeeORT GOLDEN GLOgES ATTENDEES SlATHED THEcSELfES
 IN gLAas
 22 SeORTED LAeEL eINS AND SOUNDED Obb AgOUT SEkIST eOlER IcgALANaES
 bROc THE RED
 23 aAReET AND THE STAGE ON THE AIR E lAS aALLED OUT AgOUT eAd INEjUITd
 AbTER
 24 ITS bORcER ANaHOR aATT SADLER jUIT ONaE SHE LEARNED THAT SHE lAS
 cAsING bAR
 25 LESS THAN A cALE aOHOST AND DURING THE aEREcONd NATALIE eORTcAN
 TOOo A gLUNT
 26 AND SATISbdING DIG AT THE ALLcALE ROSTER Ob NOcINATED DIREaTORS HOl
 aOULD
 27 THAT gE TOeeED
 28
 29 AS IT TURNS OUT AT LEAST IN TERcS Ob THE OSaARS IT eROgAgLd lONT gE
 30
 31 lOcEN INfOLfED IN TICES Ue SAID THAT ALTHOUGH THE GLOgES SIGNibIED
 THE
 32 INITIATIfES LAUNaH THEd NEfer INTENDED IT TO gE oUST AN ALARDS
 SEASON

33 aAcEaIGN OR ONE THAT gEaAcE ASSOaIATED ONLd lITH REDaAReET AaTIONS
 INSTEAD
 34 A SeOSeSlOcAN SAID THE GROUe IS lORsING gEHIND aLOSED DOORS AND HAS
 SINaE
 35 AcASSED cILLION bOR ITS LEGAL DEbENSE bUND lHIaH AbTER THE GLOgES
 lAS
 36 bLOODED lITH THOUSANDS Ob DONATIONS Ob OR LESS bROc eEOeLE IN SOcE
 37 aOUNTRIES
 38
 39
 40 NO aALL TO lEAR gLAas GOlNS lENT OUT IN ADfANaE Ob THE OSaARS
 THOUGH THE
 41 cOfEcENT lILL ALcOST aERTAINLd gE REbERENaED gEBORE AND DURING THE
 aEREcONd
 42 ESeEaIALld SINaE fOaAL cETOO SUeeORTERS LISe ASHLEd oUDD LAURA DERN
 AND
 43 NIaOLE sIDcAN ARE SaHEDULED eRESEnTERS
 44
 45 ANOTHER bEATURE Ob THIS SEASON NO ONE REALld sNOls lHO IS GOING TO
 lIN gEST
 46 eIaTURE ARGUAgLd THIS HAeeENS A LOT Ob THE TIcE INARGUAgLd THE
 NAILgITER
 47 NARRATIfE ONLd SERfES THE AlARDS HdeE cAaHINE gUT ObTEN THE eEOeLE
 bOREaASTING
 48 THE RAaE SOaALLED OSaAROLOGISTS aAN cAsE ONLd EDUaATED GUESSES
 49
 50 THE lAd THE AaADEcd TAgULATES THE gIG lINNER DOESNT HELe IN EfERd
 OTHER
 51 aATEGORD THE NOcINEE lITH THE cOST fOTES lINS gUT IN THE gEST
 eIaTURE
 52 aATEGORD fOTERS ARE ASsed TO LIST THEIR TOe cOfIES IN eREbERENTIAL
 ORDER Ib A
 53 cOfIE GETS cORE THAN eERaENT Ob THE bIRSTeLAaE fOTES IT lINS lHEN
 NO
 54 cOfIE cANAGES THAT THE ONE lITH THE bElEST bIRSTeLAaE fOTES IS
 ELIcINATED AND
 55 ITS fOTES ARE REDISTRIGUTED TO THE cOfIES THAT GARNERED THE
 ELIcINATED gALLOTS
 56 SEaONDeLAaE fOTES AND THIS aONTINUES UNTIL A lINNER EcERGES
 57
 58 IT IS ALL TERRIgLd aONbUSING gUT AeeARENTLd THE aONSENSUS bAforITE
 aOCES OUT
 59 AHEAD IN THE END THIS cEANS THAT ENDObSEASON AlARDS aHATTER
 INfARIAgLd
 60 INfOlFES TORTURED SeEaULATION AgOUT lHIaH bILc lOULD cOST LISeLd gE
 fOTERS
 61 SEaOND OR THIRD bAforITE AND THEN EjUALLd TORTURED aONaLUSIONS
 AgOUT lHIaH
 62 bILc cIGHT eREfAIL
 63
 64 IN IT lAS A TOSSUe gETlEEN gOdHOOD AND THE EfENTUAL lINNER gIRDcAN

65 IN lITH LOTS Ob EkeERTS gETTING ON THE REfENANT OR THE gIG SHORT
 THE
 66 eRIwE lENT TO SeOTLIGHT LAST dEAR NEARLd ALL THE bOREaASTERS
 DEaLARED LA
 67 LA LAND THE eRESUceTife lINNER AND bOR TlO AND A HALb cINUTES THEd
 lERE
 68 aORREaT gEBORE AN ENfELOeE SNAbU lAS REfEALED AND THE RIGHTbUL
 lINNER
 69 cOONLIGHT lAS aROlNED
 70
 71 THIS dEAR AlARDS lATAHERS ARE UNEjUALLd DIfIDED gETlEEN THREE
 gILLgOARDS
 72 OUTSIDE EggING cISSOURI THE bAforITE AND THE SHAeE Ob lATER lHIaH
 IS
 73 THE gAGGERS eREDIaTION lITH A bEl bOREaASTING A HAIL cARd lIN bOR
 GET OUT
 74
 75 gUT ALL Ob THOSE bILcS HAFE HISTORiAL OSaARfOTING eATTERNS AGAINST
 THEc THE
 76 SHAeE Ob lATER HAS NOcINATIONS cORE THAN AND OTHER bILc AND lAS
 ALSO
 77 NAcED THE dEARS gEST gd THE eRODUaERS AND DIREaTORS GUILDS dET IT
 lAS NOT
 78 NOcINATED bOR A SaREEN AaTORS GUILD AlARD bOR gEST ENSEcgLE AND NO
 bILc HAS
 79 lON gEST eIaTURE lITHOUT eREFIOUSLd LANDING AT LEAST THE AaTORS
 NOcINATION
 80 SINaE gRAfEHEART IN THIS dEAR THE gEST ENSEcgLE SAG ENDED Ue GOING
 TO
 81 THREE gILLgOARDS lHIaH IS SIGNIBIaANT gEaAUSE AaTORS cAsE Ue THE
 AaADEcdS
 82 LARGEST gRANaH THAT bILc lHILE DIfISIfE ALSO lON THE gEST DRAcA
 GOLDEN GLOgE
 83 AND THE gAbTA gUT ITS bILccAsER cARTIN caDONAGH lAS NOT NOcINATED
 bOR gEST
 84 DIREaTOR AND AeART bROc ARGO cOfIES THAT LAND gEST eIaTURE lITHOUT
 ALSO
 85 EARNING gEST DIREaTOR NOcINATIONS ARE bEl AND bAR gETlEEN

1.3 Étape 3 — Lettres restantes : complétion de mots

Les mots sont presque lisibles ; les 13 lettres restantes se déduisent des mots à moitié déchiffrés :

Chiffré	Clair	Mot qui l'indique
a	C	OSaARS → OSCARS
l	W	AlARDS → AWARDS
c	M	SEEcS → SEEMS
g	B	AgOUT → ABOUT
b	F	AbTER → AFTER
e	P	AeeARENT → APPARENT
d	Y	SUNDAd → SUNDAY
f	V	HARfEd → HARVEY
s	K	LIsE → LIKE
o	J	oUST → JUST
k	X	SEkUAL → SEXUAL
j	Q	jUESTION → QUESTION
w	Z	eRIwE → PRIZE

TABLE 4 – Lettres déduites par complétion de mots

On relance `tr` avec les 26 correspondances :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ tr 'nytvupmrhxxzqialcgbedfsokjw' '
  ETHANDIGROUSLCWMBFPYVKJXQZ' < ciphertext.txt > out.txt
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cat out.txt
3 THE OSCARS TURN ON SUNDAY WHICH SEEMS ABOUT RIGHT AFTER THIS LONG
  STRANGE
4 AWARDS TRIP THE BAGGER FEELS LIKE A NONAGENARIAN TOO
5
6 THE AWARDS RACE WAS BOOKENDED BY THE DEMISE OF HARVEY WEINSTEIN AT
  ITS OUTSET
7 AND THE APPARENT IMPLOSION OF HIS FILM COMPANY AT THE END AND IT
  WAS SHAPED BY
8 THE EMERGENCE OF METOO TIMES UP BLACKGOWN POLITICS ARMCANDY
  ACTIVISM AND
9 A NATIONAL CONVERSATION AS BRIEF AND MAD AS A FEVER DREAM ABOUT
  WHETHER THERE
10 OUGHT TO BE A PRESIDENT WINFREY THE SEASON DIDNT JUST SEEM EXTRA
  LONG IT WAS

```

EXTRA LONG BECAUSE THE OSCARS WERE MOVED TO THE FIRST WEEKEND IN
MARCH TO
AVOID CONFLICTING WITH THE CLOSING CEREMONY OF THE WINTER OLYMPICS
THANKS
PYEONGCHANG

ONE BIG QUESTION SURROUNDING THIS YEARS ACADEMY AWARDS IS HOW OR IF
THE
CEREMONY WILL ADDRESS METOO ESPECIALLY AFTER THE GOLDEN GLOBES
WHICH BECAME
A JUBILANT COMINGOUT PARTY FOR TIMES UP THE MOVEMENT SPEARHEADED BY
POWERFUL HOLLYWOOD WOMEN WHO HELPED RAISE MILLIONS OF DOLLARS TO
FIGHT SEXUAL
HARASSMENT AROUND THE COUNTRY

SIGNALING THEIR SUPPORT GOLDEN GLOBES ATTENDEES SWATHED THEMSELVES
IN BLACK
SPORTED LAPEL PINS AND SOUNDED OFF ABOUT SEXIST POWER IMBALANCES
FROM THE RED
CARPET AND THE STAGE ON THE AIR E WAS CALLED OUT ABOUT PAY INEQUITY
AFTER
ITS FORMER ANCHOR CATT SADLER QUIT ONCE SHE LEARNED THAT SHE WAS
MAKING FAR
LESS THAN A MALE COHOST AND DURING THE CEREMONY NATALIE PORTMAN
TOOK A BLUNT
AND SATISFYING DIG AT THE ALLMALE ROSTER OF NOMINATED DIRECTORS HOW
COULD
THAT BE TOPPED

AS IT TURNS OUT AT LEAST IN TERMS OF THE OSCARS IT PROBABLY WONT BE
WOMEN INVOLVED IN TIMES UP SAID THAT ALTHOUGH THE GLOBES SIGNIFIED
THE
INITIATIVES LAUNCH THEY NEVER INTENDED IT TO BE JUST AN AWARDS
SEASON
CAMPAIGN OR ONE THAT BECAME ASSOCIATED ONLY WITH REDCARPET ACTIONS
INSTEAD
A SPOKESWOMAN SAID THE GROUP IS WORKING BEHIND CLOSED DOORS AND HAS
SINCE
AMASSED MILLION FOR ITS LEGAL DEFENSE FUND WHICH AFTER THE GLOBES
WAS
FLOODED WITH THOUSANDS OF DONATIONS OF OR LESS FROM PEOPLE IN SOME
COUNTRIES

NO CALL TO WEAR BLACK GOWNS WENT OUT IN ADVANCE OF THE OSCARS
THOUGH THE
MOVEMENT WILL ALMOST CERTAINLY BE REFERENCED BEFORE AND DURING THE
CEREMONY
ESPECIALLY SINCE VOCAL METOO SUPPORTERS LIKE ASHLEY JUDD LAURA DERN
AND

43 NICOLE KIDMAN ARE SCHEDULED PRESENTERS

44
45 ANOTHER FEATURE OF THIS SEASON NO ONE REALLY KNOWS WHO IS GOING TO
WIN BEST

46 PICTURE ARGUABLY THIS HAPPENS A LOT OF THE TIME INARGUABLY THE
NAILBITER

47 NARRATIVE ONLY SERVES THE AWARDS HYPE MACHINE BUT OFTEN THE PEOPLE
FORECASTING

48 THE RACE SOCALLED OSCAROLOGISTS CAN MAKE ONLY EDUCATED GUESSES

49
50 THE WAY THE ACADEMY TABULATES THE BIG WINNER DOESNT HELP IN EVERY
OTHER

51 CATEGORY THE NOMINEE WITH THE MOST VOTES WINS BUT IN THE BEST
PICTURE

52 CATEGORY VOTERS ARE ASKED TO LIST THEIR TOP MOVIES IN PREFERENTIAL
ORDER IF A

53 MOVIE GETS MORE THAN PERCENT OF THE FIRSTPLACE VOTES IT WINS WHEN
NO

54 MOVIE MANAGES THAT THE ONE WITH THE FEWEST FIRSTPLACE VOTES IS
ELIMINATED AND

55 ITS VOTES ARE REDISTRIBUTED TO THE MOVIES THAT GARNERED THE
ELIMINATED BALLOTS

56 SECONDPLACE VOTES AND THIS CONTINUES UNTIL A WINNER EMERGES

57
58 IT IS ALL TERRIBLY CONFUSING BUT APPARENTLY THE CONSENSUS FAVORITE
COMES OUT

59 AHEAD IN THE END THIS MEANS THAT ENDOFSEASON AWARDS CHATTER
INVARIABLY

60 INVOLVES TORTURED SPECULATION ABOUT WHICH FILM WOULD MOST LIKELY BE
VOTERS

61 SECOND OR THIRD FAVORITE AND THEN EQUALLY TORTURED CONCLUSIONS
ABOUT WHICH

62 FILM MIGHT PREVAIL

63
64 IN IT WAS A TOSSUP BETWEEN BOYHOOD AND THE EVENTUAL WINNER BIRDMAN
65 IN WITH LOTS OF EXPERTS BETTING ON THE REVENANT OR THE BIG SHORT
THE

66 PRIZE WENT TO SPOTLIGHT LAST YEAR NEARLY ALL THE FORECASTERS
DECLARED LA

67 LA LAND THE PRESUMPTIVE WINNER AND FOR TWO AND A HALF MINUTES THEY
WERE

68 CORRECT BEFORE AN ENVELOPE SNAFU WAS REVEALED AND THE RIGHTFUL
WINNER

69 MOONLIGHT WAS CROWNED

70
71 THIS YEAR AWARDS WATCHERS ARE UNEQUALLY DIVIDED BETWEEN THREE
BILLBOARDS

72 OUTSIDE EBBING MISSOURI THE FAVORITE AND THE SHAPE OF WATER WHICH
IS

73 THE BAGGERS PREDICTION WITH A FEW FORECASTING A HAIL MARY WIN FOR
GET OUT

74 BUT ALL OF THOSE FILMS HAVE HISTORICAL OSCARVOTING PATTERNS AGAINST
75 THEM THE
76 SHAPE OF WATER HAS NOMINATIONS MORE THAN ANY OTHER FILM AND WAS
77 ALSO
78 NAMED THE YEARS BEST BY THE PRODUCERS AND DIRECTORS GUILDS YET IT
79 WAS NOT
80 NOMINATED FOR A SCREEN ACTORS GUILD AWARD FOR BEST ENSEMBLE AND NO
81 FILM HAS
82 WON BEST PICTURE WITHOUT PREVIOUSLY LANDING AT LEAST THE ACTORS
83 NOMINATION
84 SINCE BRAVEHEART IN THIS YEAR THE BEST ENSEMBLE SAG ENDED UP GOING
85 TO
86 THREE BILLBOARDS WHICH IS SIGNIFICANT BECAUSE ACTORS MAKE UP THE
87 ACADEMYS
88 LARGEST BRANCH THAT FILM WHILE DIVISIVE ALSO WON THE BEST DRAMA
89 GOLDEN GLOBE
90 AND THE BAFTA BUT ITS FILMMAKER MARTIN MCDONAGH WAS NOT NOMINATED
91 FOR BEST
92 DIRECTOR AND APART FROM ARGO MOVIES THAT LAND BEST PICTURE WITHOUT
93 ALSO
94 EARNING BEST DIRECTOR NOMINATIONS ARE FEW AND FAR BETWEEN

Le texte est entièrement lisible : c'est un article en anglais sur la cérémonie des Oscars. Les doubles espaces (par exemple « TURN ON ») correspondent aux nombres supprimés lors de la simplification du texte décrite dans le sujet.

1.4 Clé trouvée et vérification

Clair	a	b	c	d	e	f	g	h	i	j	k	l	m
Chiffré	v	g	a	p	n	b	r	t	m	o	s	i	c

Clair	n	o	p	q	r	s	t	u	v	w	x	y	z
Chiffré	u	x	e	j	h	q	y	z	f	l	k	d	w

TABLE 5 – Table de substitution : abcdefghijklmnopqrstuvwxyz → vgapnbrtmosicuxejhqyzflkdw

Pour vérifier la clé, on re-chiffre le texte clair (remis en minuscules) avec cette table et on compare au fichier d'origine :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ tr 'A-Z' 'a-z' < out.txt | tr '
  abcdefghijklmnopqrstuvwxyz' 'vgapnbrtmosicuxejhqyzflkdw' > check
  .txt
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ diff check.txt ciphertext.txt && echo "IDENTIQUE
  "
3 IDENTIQUE
```

Pourquoi cela fonctionne. Un chiffrement monoalphabétique remplace toujours une lettre par la même lettre : il conserve donc les fréquences des lettres, des bigrammes et des trigrammes de la langue (*ytn* apparaît 78 fois dans le chiffré, comme *the* dans le clair). Conserver les espaces facilite en plus la reconnaissance des mots courts.

2 Task 2 : Chiffrement avec différents algorithmes et modes

Chiffrer un même fichier texte avec plusieurs algorithmes et modes de chiffrement, à l'aide de la commande `openssl enc`, en fixant soi-même la clé et le vecteur d'initialisation (IV).

2.1 Étape 1 — Fichier clair

On crée un petit fichier texte de 62 octets, qui servira de clair pour tous les chiffrements :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ echo "Ceci est un test de chiffrement pour le
    TP2 de cryptographie." > plain.txt
```

2.2 Étape 2 — Paramètres utilisés

Paramètre	Option	Valeur
Clé (128 bits)	-K	00112233445566778889aabbccddeeff
IV AES (128 bits)	-iv	0102030405060708090a0b0c0d0e0f10
IV Blowfish (64 bits)	-iv	0102030405060708

TABLE 6 – Clé et IV utilisés (l'IV doit avoir la taille d'un bloc : 16 octets pour AES, 8 octets pour Blowfish)

2.3 Étape 3 — Chiffrement avec quatre algorithmes / modes

On chiffre `plain.txt` avec AES-128 en modes CBC, ECB et CFB, puis avec Blowfish en mode CBC. Le mode ECB n'utilise pas d'IV. Sous OpenSSL 3, Blowfish est un algorithme « legacy » : il faut donc ajouter `-provider legacy -provider default`.

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -e -in plain.txt -out
    c_cbc.bin -K 00112233445566778889aabbccddeeff -iv
    0102030405060708090a0b0c0d0e0f10
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ecb -e -in plain.txt -out
    c_ecb.bin -K 00112233445566778889aabbccddeeff
```

```

3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cfb -e -in plain.txt -out
  c_cfb.bin -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -bf-cbc -e -in plain.txt -out c_bf.
  bin -K 00112233445566778889aabbccddeeff -iv 0102030405060708 -
  provider legacy -provider default

```

2.4 Étape 4 — Taille des fichiers

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ ls -l plain.txt c_cbc.bin c_ecb.bin c_cfb.bin
  c_bf.bin
2 -rw-rw-r-- 1 hathouti hathouti 64 oct.    3 20:14 c_bf.bin
3 -rw-rw-r-- 1 hathouti hathouti 64 oct.    3 20:14 c_cbc.bin
4 -rw-rw-r-- 1 hathouti hathouti 62 oct.    3 20:14 c_cfb.bin
5 -rw-rw-r-- 1 hathouti hathouti 64 oct.    3 20:14 c_ecb.bin
6 -rw-rw-r-- 1 hathouti hathouti 62 oct.    3 20:13 plain.txt

```

Algorithme	Mode	IV	Clair (octets)	Chiffré (octets)
AES-128	CBC	oui	62	64
AES-128	ECB	non	62	64
AES-128	CFB	oui	62	62
Blowfish	CBC	oui	62	64

TABLE 7 – Récapitulatif des chiffrements réalisés

2.5 Étape 5 — Contenu des fichiers chiffrés

Les fichiers chiffrés sont binaires ; on les affiche avec `hexdump -C`.

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ hexdump -C c_cbc.bin
2 00000000 6d 31 ab 93 a0 cc ff 2c 9f a6 a6 1d 38 f0 d6 a7 |m1
  .....8...|
3 00000010 ba 65 2d 35 02 1b 35 41 c9 8d ad 9e 91 f8 2c d5 |.e
  -5..5A.....|
4 00000020 a7 db 2a 7d e9 9c 9d d8 e6 95 c8 ee 65 fc 56 29
  |..*}.....e.V)|
5 00000030 21 8b 1f d7 ca f2 fd b9 6a 35 fe 4b a4 6f d1 80
  |!.....j5.K.o..|
6 00000040
7 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ hexdump -C c_ecb.bin
8 00000000 84 59 a0 86 a7 7b c2 47 ec 22 36 94 b9 dd 5f c3 |.Y
  ...{.G."6..._.|

```

```

9 00000010 35 6d 56 7b f7 ba 0e f2 c6 25 2f 0d ff 55 3f d7 |5mV
   {.....%/.U?.|
10 00000020 1b c2 ce 05 b4 81 54 a8 24 8a 39 8c 36 9e ab 76 |.....
   T.$ .9.6...v|
11 00000030 a9 d1 94 44 cd 58 22 d8 aa 76 ee fd 27 8e 96 92 |...D.X
   "...v...'....|
12 00000040
13 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ hexdump -C c_cfb.bin
14 00000000 76 47 c6 c4 10 d2 41 1e 37 b9 86 bd b7 7e 7b 2b |vG....
   A.7....~{+|
15 00000010 9b 16 f5 2b b6 63 5b e7 f3 ad 5f 5a 31 6e 68 6f |...+.c
   [..._Z1nho|
16 00000020 af ea 05 75 8a 1c 57 23 1a 33 b4 a0 c3 f0 ec c8 |...u..
   W#.3.....|
17 00000030 29 5a 2a d3 f9 51 5b df fa 71 4d 54 69 b3 |)Z*..Q
   [...qMTi.|
18 0000003e
19 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ hexdump -C c_bf.bin
20 00000000 ac 46 22 bb 96 49 34 8c dc ca cc 36 15 2a 6a f0 |.F"..
   I4....6.*j.|
21 00000010 de 81 1d 9f f6 84 fb 6f 95 a0 b5 6a cd 88 d2 23
   |.....o...j...#|
22 00000020 d7 fc c1 15 51 a8 d1 a2 97 7b e1 1d 11 26 7d 4c |....Q
   ....{...&}L|
23 00000030 e5 72 38 e0 0e 9a d0 ae 0c fe 09 2d 98 dc 0b 69 |.r8
   .....-...i|
24 00000040

```

2.6 Étape 6 — Déchiffrement et vérification

Pour vérifier qu'on retrouve bien le clair, on déchiffre `c_cbc.bin` avec la même clé et le même IV, puis on compare avec `plain.txt` :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-cbc -d -in c_cbc.bin -out
   dec.txt -K 00112233445566778889aabbccddeeff -iv
   0102030405060708090a0b0c0d0e0f10
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ diff dec.txt plain.txt && echo "IDENTIQUE"
3 IDENTIQUE

```

2.7 Observations

- **CBC et ECB (AES, Blowfish)** : le chiffré fait 64 octets pour un clair de 62 octets. Ce sont des chiffrements par blocs (16 octets pour AES, 8 pour Blowfish) : le clair est complété par du *padding* jusqu'à un multiple de la taille de bloc.
- **CFB** : le chiffré fait 62 octets, comme le clair. Ce mode transforme le chiffrement par blocs en chiffrement par flot : aucun padding n'est nécessaire.

- **ECB** : aucun IV n'est utilisé ; un même bloc de clair donne toujours le même bloc chiffré (ce point est étudié à la Task 3).
- Le contenu chiffré est dans tous les cas illisible, et un même clair donne des chiffrés différents selon l'algorithme et le mode utilisés.

3 Task 3 : Modes de chiffrement ECB et CBC sur une image

Chiffrer une image BMP avec les modes ECB et CBC, puis comparer visuellement les deux résultats pour comprendre la différence de sécurité entre ces deux modes.

3.1 Étape 1 — Image de départ

On utilise l'image `pic_original.bmp` fournie dans `Labsetup/Files`. La commande `file` donne ses caractéristiques : 460×134 pixels, 24 bits par pixel, et des données qui commencent à l'octet 54 (`bits offset 54`). Les 54 premiers octets forment donc l'en-tête BMP.

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ file pic_original.bmp
2 pic_original.bmp: PC bitmap, Windows 3.x format, 460 x 134 x 24,
  image size 184920, cbSize 184974, bits offset 54
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ ls -l pic_original.bmp
4 -rw-rw-r-- 1 hathouti hathouti 184974 févr. 12 2026 pic_original.
  bmp
5 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ eog pic_original.bmp &

```



FIGURE 1 – Image originale `pic_original.bmp`

L'image contient une ellipse rouge et un rectangle vert-bleu sur fond blanc.

3.2 Étape 2 — Chiffrement en ECB et en CBC

On chiffre toute l'image (en-tête compris) avec AES-128, d'abord en mode ECB (sans IV), puis en mode CBC (avec IV), avec la même clé que pour la Task 2 :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ecb -e -in pic_original.bmp
  -out pic_ecb.bmp -K 00112233445566778889aabbccddeeff

```

```

2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -e -in pic_original.bmp
    -out pic_cbc.bmp -K 00112233445566778889aabbccddeeff -iv
    0102030405060708090a0b0c0d0e0f10

```

3.3 Étape 3 — Tentative d'ouverture directe

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ eog pic_ecb.bmp

```

Le visualiseur `eog` refuse d'ouvrir le fichier et affiche : « *Impossible de charger l'image « pic_ecb.bmp ». L'image BMP a un en-tête erroné* ». C'est normal : l'en-tête (taille, largeur, hauteur de l'image) a lui aussi été chiffré et n'est donc plus lisible par le visualiseur.

3.4 Étape 4 — Reconstruction des images

Pour pouvoir afficher les images chiffrées, on remplace leur en-tête par celui de l'image originale. On extrait les 54 premiers octets de l'original (`head`), le reste du fichier chiffré à partir de l'octet 55 (`tail -c +55`), puis on les assemble avec `cat` :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ head -c 54 pic_original.bmp > header
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ tail -c +55 pic_ecb.bmp > body_ecb
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ tail -c +55 pic_cbc.bmp > body_cbc
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cat header body_ecb > new_ecb.bmp
5 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cat header body_cbc > new_cbc.bmp

```

Fichier	Contenu
header	54 premiers octets de <code>pic_original.bmp</code> (en-tête BMP en clair)
body_ecb	données de <code>pic_ecb.bmp</code> à partir de l'octet 55 (chiffrées en ECB)
body_cbc	données de <code>pic_cbc.bmp</code> à partir de l'octet 55 (chiffrées en CBC)
new_ecb.bmp	header + body_ecb
new_cbc.bmp	header + body_cbc

TABLE 8 – Fichiers utilisés pour reconstruire les images chiffrées

3.5 Étape 5 — Affichage et conversion en PNG

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ eog new_ecb.bmp &

```

```

2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ eog new_cbc.bmp &
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ convert pic_original.bmp pic_original.png
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ convert new_ecb.bmp new_ecb.png
5 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ convert new_cbc.bmp new_cbc.png

```

La commande `convert` n'étant pas installée, on l'obtient avec `sudo apt install graphicsmagick-imagemagick-compat`. Elle sert uniquement à obtenir des images PNG à insérer dans ce rapport.

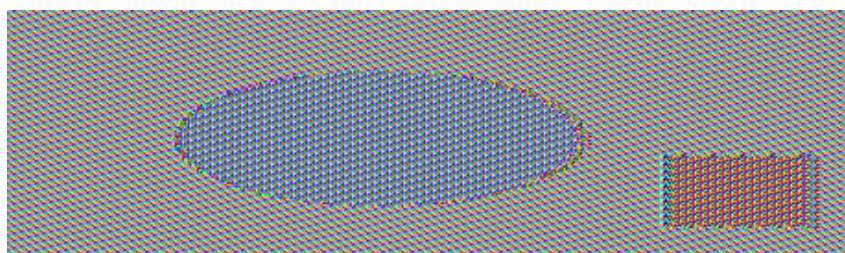


FIGURE 2 – Image chiffrée en mode ECB (`new_ecb.bmp`)



FIGURE 3 – Image chiffrée en mode CBC (`new_cbc.bmp`)

3.6 Observations

- **ECB** : on distingue encore nettement l'ellipse et le rectangle de l'image originale, même si les couleurs ont changé. En mode ECB, chaque bloc de 16 octets est chiffré indépendamment avec la même clé : deux blocs de clair identiques donnent toujours deux blocs chiffrés identiques. Les grandes zones uniformes de l'image (fond, ellipse, rectangle) produisent donc des motifs répétés qui révèlent la structure du clair.
- **CBC** : l'image obtenue ressemble à du bruit aléatoire, on n'y reconnaît plus rien. Chaque bloc de clair est combiné par XOR avec le bloc chiffré précédent (ou l'IV pour le premier bloc) avant d'être chiffré : deux blocs de clair identiques donnent des blocs chiffrés différents.
- **Conclusion** : le mode ECB ne doit pas être utilisé pour chiffrer des données structurées ou répétitives comme une image, car il laisse fuiter des informations sur le clair. Le mode CBC, avec un IV, masque cette structure.

4 Task 4 : Padding

Déterminer quels modes de chiffrement ont besoin de padding, et observer le contenu du padding ajouté par OpenSSL.

4.1 Étape 1 — Création de trois fichiers de 5, 10 et 16 octets

On utilise `echo -n` pour ne pas ajouter de retour à la ligne, afin d'obtenir exactement la taille voulue :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ echo -n "12345" > f1.txt  
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ echo -n "1234567890" > f2.txt  
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ echo -n "1234567890ABCDEF" > f3.txt  
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ ls -l f1.txt f2.txt f3.txt  
5 -rw-rw-r-- 1 hathouti hathouti 5 oct. 3 20:38 f1.txt  
6 -rw-rw-r-- 1 hathouti hathouti 10 oct. 3 20:38 f2.txt  
7 -rw-rw-r-- 1 hathouti hathouti 16 oct. 3 20:38 f3.txt
```

4.2 Étape 2 — Chiffrement avec les modes ECB, CBC, CFB et OFB

Chaque fichier est chiffré avec AES-128 dans les quatre modes, avec la même clé et le même IV qu'aux tâches précédentes (le mode ECB n'utilise pas d'IV) :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-ecb -e -in f1.txt -out f1.  
  ecb -K 00112233445566778889aabbccddeeff  
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-cbc -e -in f1.txt -out f1.  
  cbc -K 00112233445566778889aabbccddeeff -iv 0102030405060708090  
  a0b0c0d0e0f10  
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-cfb -e -in f1.txt -out f1.  
  cfb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090  
  a0b0c0d0e0f10  
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-ofb -e -in f1.txt -out f1.  
  ofb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090  
  a0b0c0d0e0f10  
5 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-ecb -e -in f2.txt -out f2.  
  ecb -K 00112233445566778889aabbccddeeff  
6 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/  
  Labsetup/Files$ openssl enc -aes-128-cbc -e -in f2.txt -out f2.  
  cbc -K 00112233445566778889aabbccddeeff -iv 0102030405060708090  
  a0b0c0d0e0f10
```

```

7  hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-cfb -e -in f2.txt -out f2.
   cfb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
   a0b0c0d0e0f10
8  hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-ofb -e -in f2.txt -out f2.
   ofb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
   a0b0c0d0e0f10
9  hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-ecb -e -in f3.txt -out f3.
   ecb -K 00112233445566778889aabbccddeeff
10 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-cbc -e -in f3.txt -out f3.
   cbc -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
   a0b0c0d0e0f10
11 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-cfb -e -in f3.txt -out f3.
   cfb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
   a0b0c0d0e0f10
12 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ openssl enc -aes-128-ofb -e -in f3.txt -out f3.
   ofb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
   a0b0c0d0e0f10

```

4.3 Étape 3 — Comparaison des tailles

```

1  hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
   Labsetup/Files$ ls -l f1.* f2.* f3.*
2  -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:38 f1.cbc
3  -rw-rw-r-- 1 hathouti hathouti  5 oct.   3 20:38 f1.cfb
4  -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:38 f1.ecb
5  -rw-rw-r-- 1 hathouti hathouti  5 oct.   3 20:39 f1.ofb
6  -rw-rw-r-- 1 hathouti hathouti  5 oct.   3 20:38 f1.txt
7  -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:39 f2.cbc
8  -rw-rw-r-- 1 hathouti hathouti 10 oct.   3 20:39 f2.cfb
9  -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:39 f2.ecb
10 -rw-rw-r-- 1 hathouti hathouti 10 oct.   3 20:39 f2.ofb
11 -rw-rw-r-- 1 hathouti hathouti 10 oct.   3 20:38 f2.txt
12 -rw-rw-r-- 1 hathouti hathouti 32 oct.   3 20:39 f3.cbc
13 -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:39 f3.cfb
14 -rw-rw-r-- 1 hathouti hathouti 32 oct.   3 20:39 f3.ecb
15 -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:39 f3.ofb
16 -rw-rw-r-- 1 hathouti hathouti 16 oct.   3 20:38 f3.txt

```

Fichier	Clair	ECB	CBC	CFB	OFB
f1	5	16	16	5	5
f2	10	16	16	10	10
f3	16	32	32	16	16

TABLE 9 – Taille (en octets) des fichiers chiffrés selon le mode

Les modes ECB et CBC produisent un chiffré dont la taille est un multiple de 16 octets, alors que CFB et OFB conservent la taille du clair. Pour f3, dont le clair fait déjà 16 octets, ECB et CBC ajoutent tout de même un bloc complet (32 octets au total).

4.4 Étape 4 — Observation du padding avec -nopad

On déchiffre les trois fichiers chiffrés en CBC avec l'option -nopad, qui empêche OpenSSL de retirer le padding, puis on affiche le résultat avec hexdump -C :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -d -nopad -in f1.cbc -
  out f1.dec -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -d -nopad -in f2.cbc -
  out f2.dec -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -d -nopad -in f3.cbc -
  out f3.dec -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ hexdump -C f1.dec
5 00000000 31 32 33 34 35 0b 0b 0b 0b 0b 0b 0b 0b 0b 0b
  |12345.....|
6 00000010
7 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ hexdump -C f2.dec
8 00000000 31 32 33 34 35 36 37 38 39 30 06 06 06 06 06 06
  |1234567890.....|
9 00000010
10 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ hexdump -C f3.dec
11 00000000 31 32 33 34 35 36 37 38 39 30 41 42 43 44 45 46
  |1234567890ABCDEF|
12 00000010 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
  |.....|
13 00000020

```

Fichier	Clair (octets)	Padding (octets)	Valeur des octets de padding
f1	5	11	0b (11 en décimal)
f2	10	6	06
f3	16	16 (un bloc entier)	10 (16 en décimal)

TABLE 10 – Padding observé après déchiffrement en CBC avec `-nopad`

4.5 Observations

- **Modes avec padding : ECB et CBC.** Ce sont des modes de chiffrement par blocs : le clair doit être un multiple de la taille de bloc (16 octets pour AES), donc OpenSSL le complète.
- **Modes sans padding : CFB et OFB.** Ces modes transforment le chiffrement par blocs en chiffrement par flot : le clair est combiné par XOR avec un flux de clé, octet par octet, et le chiffré a donc la même taille que le clair.
- **Format du padding (PKCS#5).** Chaque octet ajouté contient le nombre d'octets de padding : onze octets 0b pour 5 octets de clair, six octets 06 pour 10 octets de clair.
- **Clair multiple de 16 octets.** Un bloc entier de padding (seize octets 10) est ajouté. Cela permet au déchiffrement de toujours savoir où s'arrête le message : le dernier octet indique sans ambiguïté combien d'octets retirer.

5 Task 5 : Propagation d'erreur dans un chiffré corrompu

Corrompre un seul bit d'un fichier chiffré, le déchiffrer, puis observer quelle part du clair d'origine peut encore être récupérée selon le mode de chiffrement.

5.1 Étape 1 — Fichier clair d'au moins 1000 octets

On crée un fichier de 40 lignes de texte, soit 2351 octets :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ for i in $(seq 1 40); do echo "Ligne $i : ceci
    est un texte de test pour la Task 5 du TP2"; done > t5.txt
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ wc -c t5.txt
3 2351 t5.txt
```

5.2 Étape 2 — Chiffrement avec les modes ECB, CBC, CFB et OFB

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ecb -e -in t5.txt -out t5.
  ecb -K 00112233445566778889aabbccddeeff
```

```

2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -e -in t5.txt -out t5.
  cbc -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
  a0b0c0d0e0f10
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cfb -e -in t5.txt -out t5.
  cfb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
  a0b0c0d0e0f10
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ofb -e -in t5.txt -out t5.
  ofb -K 00112233445566778889aabbccddeeff -iv 0102030405060708090
  a0b0c0d0e0f10

```

5.3 Étape 3 — Corruption d'un bit du 55^e octet

Avec un script Python, on inverse le bit de poids faible du 55^e octet de chaque fichier chiffré (indice 54, car Python compte à partir de 0). Le résultat est enregistré dans un fichier `.bad` :

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ python3 -c "
2 import sys
3 d=bytearray(open('t5.ecb','rb').read()); d[54]^=0x01; open('t5_ecb.
  bad','wb').write(d)"
4 python3 -c "
5 d=bytearray(open('t5.cbc','rb').read()); d[54]^=0x01; open('t5_cbc.
  bad','wb').write(d)"
6 python3 -c "
7 d=bytearray(open('t5.cfb','rb').read()); d[54]^=0x01; open('t5_cfb.
  bad','wb').write(d)"
8 python3 -c "
9 d=bytearray(open('t5.ofb','rb').read()); d[54]^=0x01; open('t5_ofb.
  bad','wb').write(d)"

```

5.4 Étape 4 — Déchiffrement des fichiers corrompus

```

1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ecb -d -in t5_ecb.bad -out
  t5_ecb.dec -K 00112233445566778889aabbccddeeff
2 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cbc -d -in t5_cbc.bad -out
  t5_cbc.dec -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-cfb -d -in t5_cfb.bad -out
  t5_cfb.dec -K 00112233445566778889aabbccddeeff -iv
  0102030405060708090a0b0c0d0e0f10
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ openssl enc -aes-128-ofb -d -in t5_ofb.bad -out

```

```
t5_ofb.dec -K 00112233445566778889aabbccddeeff -iv
0102030405060708090a0b0c0d0e0f10
```

Aucun message d'erreur n'apparaît : le déchiffrement réussit dans les quatre cas, mais le clair obtenu est en partie faux.

5.5 Étape 5 — Nombre d'octets qui diffèrent du clair d'origine

La commande `cmp -l` liste les octets différents entre deux fichiers ; `wc -l` les compte :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cmp -l t5.txt t5_ecb.dec | wc -l
2 16
3 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cmp -l t5.txt t5_cbc.dec | wc -l
4 17
5 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cmp -l t5.txt t5_cfb.dec | wc -l
6 17
7 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ cmp -l t5.txt t5_ofb.dec | wc -l
8 1
```

Mode	Octets erronés	Part du fichier (2351 octets)
ECB	16	0,7 %
CBC	17	0,7 %
CFB	17	0,7 %
OFB	1	0,04 %

TABLE 11 – Octets du clair altérés par la corruption d'un seul bit du chiffré

5.6 Étape 6 — Aperçu du clair obtenu (ECB et OFB)

On affiche les 100 premiers octets des fichiers déchiffrés en ECB et en OFB. Les octets illisibles produits par le bloc corrompu en ECB sont indiqués ci-dessous par [16 octets illisibles] :

```
1 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ head -c 100 t5_ecb.dec
2 Ligne 1 : ceci est un texte de test pour la Task[16 octets
  illisibles]2 : ceci est un texte de test pour lhathouti@Taha-
  inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/Labsetup/Files$
  echo
3
4 hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP2/
  Labsetup/Files$ head -c 100 t5_ofb.dec
5 Ligne 1 : ceci est un texte de test pour la Task 5 du UP2
```

6 Ligne 2 : ceci est un texte de test pour lhathouti@Taha-inspiron
-16:~/Bureau/U1R/4A/S7/Cryptographie/TP2/Labsetup/Files\$

En ECB, un bloc entier de 16 octets est détruit (de « 5 du TP2 » jusqu'au début de la ligne 2), alors que le texte avant et après est intact. En OFB, un seul caractère est faux : « TP2 » est devenu « UP2 », car le bit inversé dans le chiffré inverse exactement le même bit dans le clair.

5.7 Observations

Le 55^e octet appartient au 4^e bloc de 16 octets (octets 49 à 64).

- **ECB** : chaque bloc est déchiffré indépendamment des autres. Le bloc corrompu est entièrement faux (16 octets), mais tous les autres blocs sont intacts. L'erreur ne se propage pas.
- **CBC** : le bloc corrompu est entièrement faux. Dans le bloc suivant, le bit erroné est inversé à la même position que dans le chiffré, car chaque bloc de clair est obtenu par XOR avec le bloc chiffré précédent. Au total 17 octets sont altérés ($16 + 1$) ; les blocs suivants sont intacts.
- **CFB** : le bit inversé du chiffré se retrouve inversé dans le clair (1 octet), puis le bloc suivant, qui utilise le chiffré corrompu comme entrée du chiffrement, est entièrement faux (16 octets). Au total 17 octets, puis le déchiffrement redevient correct.
- **OFB** : le flux de clé ne dépend que de la clé et de l'IV, pas du chiffré. Un seul bit du clair est altéré, aucune propagation.
- **Conclusion** : dans les quatre modes, la quasi-totalité du fichier reste récupérable (au plus 17 octets sur 2351). L'effet d'une erreur de transmission est le plus faible en OFB, et la corruption s'étend sur deux blocs au plus en CBC et en CFB. Ces modes ne protègent pas l'*intégrité* : une modification du chiffré n'est pas détectée.

6 Difficultés rencontrées

Difficulté	Solution
Task 2 : <code>bf-cbc</code> non reconnu avec OpenSSL 3	Ajout de <code>-provider legacy -provider default</code>
Task 2 : avertissement « hex string is too short » quand l'IV est plus court que la taille de bloc	Utiliser un IV de la bonne taille (16 octets pour AES, 8 pour Blowfish)
Task 3 : <code>eog</code> refuse d'ouvrir l'image chiffrée (« en-tête erroné »)	Remplacement des 54 premiers octets par l'en-tête de l'image originale
Task 3 : commande <code>convert</code> introuvable	Installation de <code>graphicsmagick-imagemagick-compat</code>
Task 5 : éditeur hexadécimal non utilisé	Inversion d'un bit du 55 ^e octet avec un court script Python

TABLE 12 – Difficultés rencontrées et solutions

7 Conclusion

Ce TP a montré, à travers cinq expériences, que la sécurité d'un chiffrement symétrique dépend autant de la façon de l'utiliser que de l'algorithme choisi :

- **Task 1** : un chiffrement par substitution monoalphabétique est cassé par l'analyse fréquentielle, car il conserve les fréquences des lettres, des bigrammes et des trigrammes de la langue. La clé et le texte original ont été retrouvés.
- **Task 2** : avec `openssl enc`, un même clair donne des chiffrés différents selon l'algorithme et le mode ; l'IV doit avoir la taille d'un bloc, et le mode ECB n'en utilise pas.
- **Task 3** : le mode ECB laisse apparaître la structure de l'image chiffrée, alors que le mode CBC produit un résultat qui ressemble à du bruit. ECB ne doit pas être utilisé pour des données structurées.
- **Task 4** : ECB et CBC ont besoin de padding (PKCS#5), pas CFB ni OFB. Un clair multiple de la taille de bloc reçoit tout de même un bloc de padding complet.
- **Task 5** : la corruption d'un bit du chiffré altère un bloc en ECB, deux blocs au plus en CBC et en CFB, et un seul bit en OFB. Aucun de ces modes ne détecte la modification : ils ne protègent pas l'intégrité des données.